Be steadfast, immovable, always abounding in the work of the Lord.
1 Corinthians 15:58

Supervised Learning Classification: Support Vector Machine

Course instructor: Prasanna Ranjith Christodoss
Associate Professor of Comp. Sc., Messiah University, PA 17055

8.3 Support Vector Classifiers

- 8.3.1 Performing SVC
- 8.3.2 Bias and Variance in SVCs

8.3 Support Vector Classifiers (continued)

- Support vectors are data points that are closer to the hyperplane and influence the position and orientation of the hyperplane.
- Support Vector Machine (SVM) is one of the most popular supervised learning algorithms.
- It is used for classification as well as regression problems.

8.3.1 Performing SVC

» There are a few practical details that go along with using SVC:
- ✓SVCs aren't naturally suited to multiclass classification.
- ✓They are typically wrapped up in either one-versus-one (OvO) or one-versus-rest (OvR) systems.

» There are four ways (actually more) that we can get an SVC in sklearn:
- ✓Using LinearSVC
- ✓Using SVC with a linear kernel
- ✓Using SVC with a polynomial kernel of degree 1 (a line in disguise)
- ✓Using a NuSVC with a linear kernel

8.3.2 Bias and Variance in SVCs

- Supervised machine learning algorithms can be understood through the lens of the bias-variance trade-off.
- Bias are the simplifying assumptions made by a model to make the target function easier to learn.
- Low bias suggests less assumptions about the form of the target function.
- High bias suggests more assumptions about the form of the target function.
- Support Vector Machines is an example of low-bias machine learning algorithm.

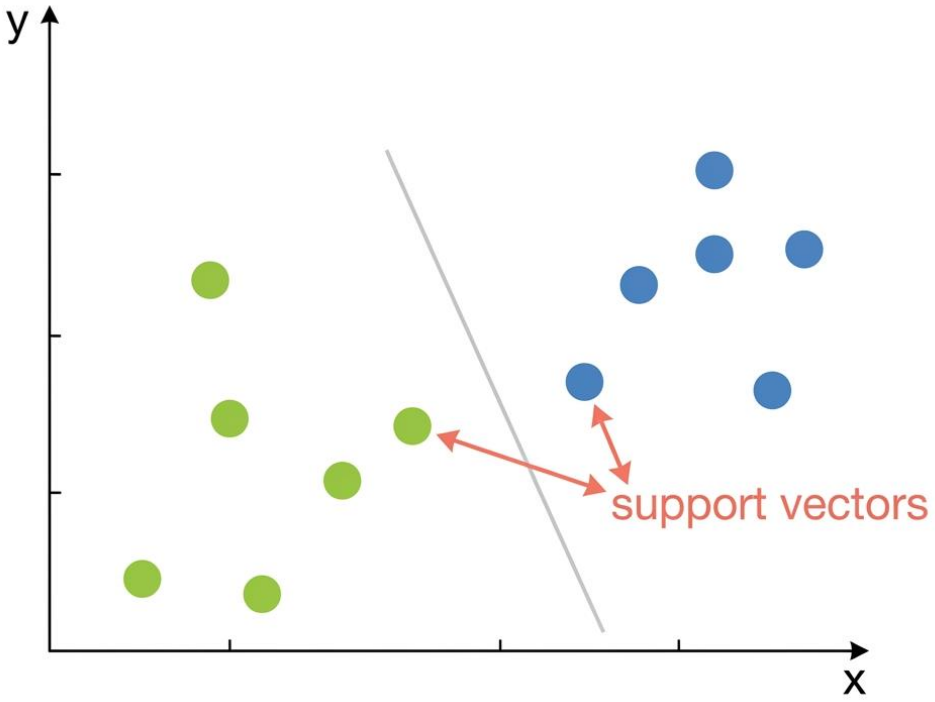
8.3.2 Bias and Variance in SVCs (continued)

- Variance is the amount that the estimate of the target function will change if different training data was used.
- Low variance suggests small changes to the estimate of the target function with changes to the training dataset.
- High variance suggests large changes to the estimate of the target function with changes to the training dataset.
- SVM is an example of high-variance machine learning algorithm.

SVM

- A supervised learning algorithm that finds the optimal separating hyperplane between labeled classes.
- Originated in the 1990s to address pattern-recognition problems with rigorous statistical foundations.
- Hyperplane, margin, and support vectors constitute the geometric core of the model.

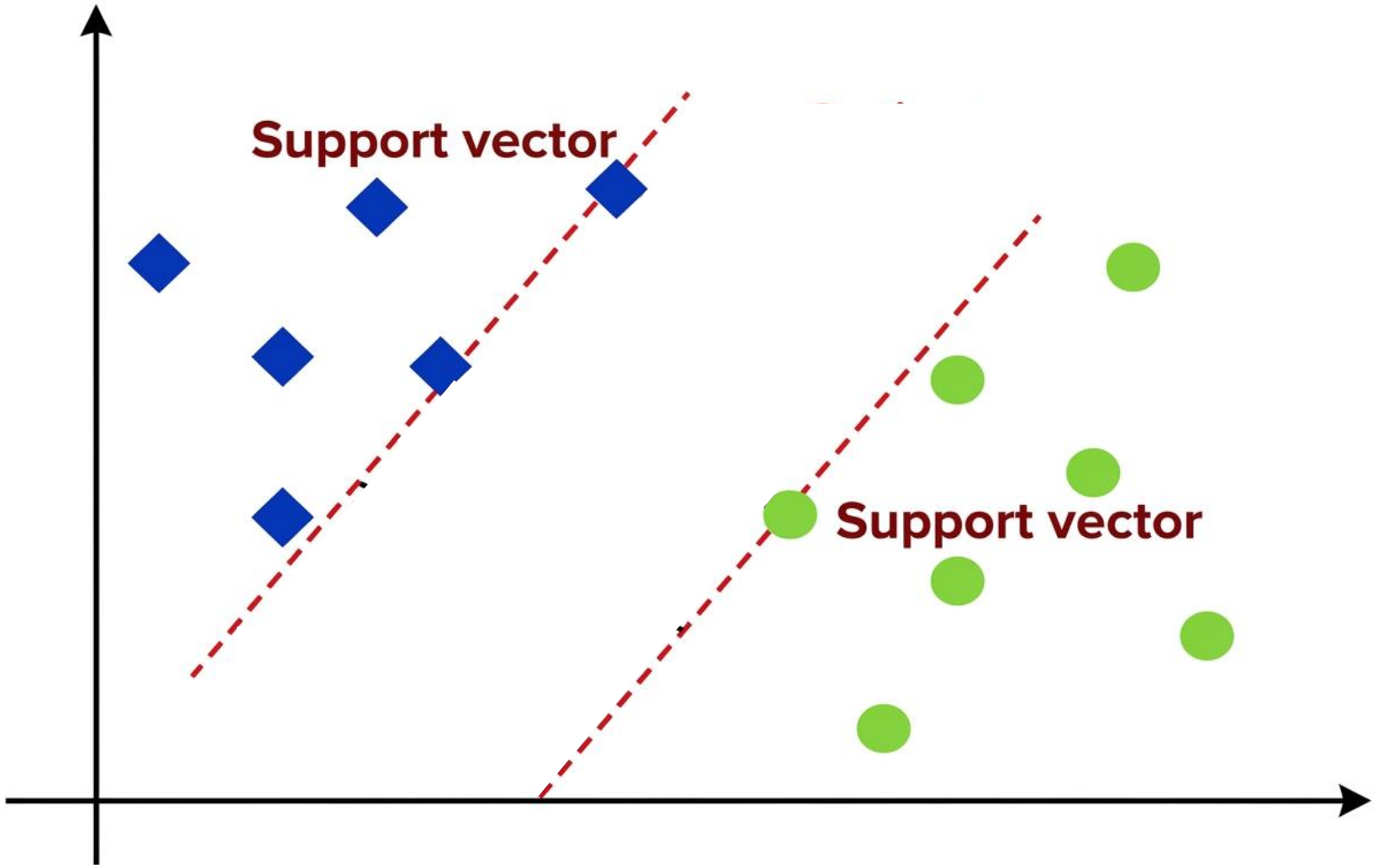
What is Support Vector?



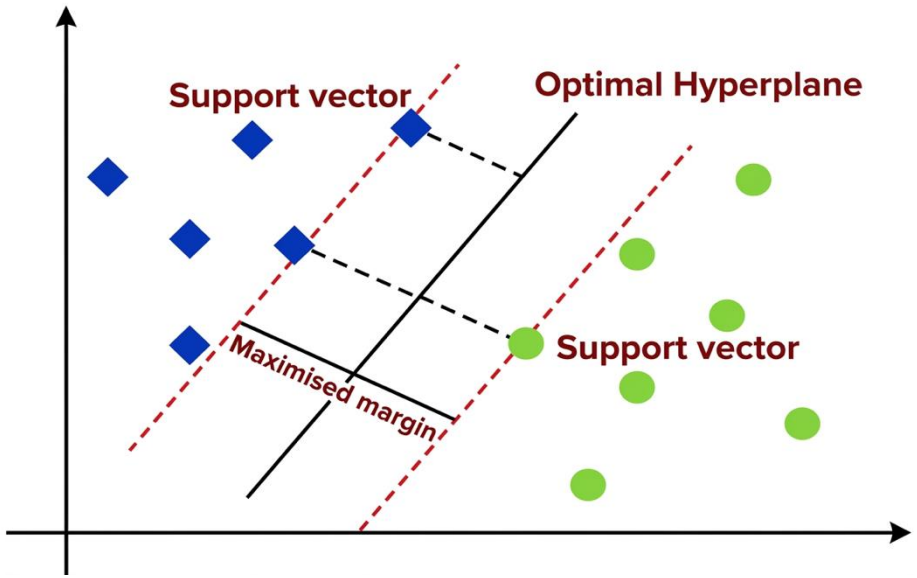
Closest points to the hyperplane are called as **support vectors**

Support Vectors Machine?
Because we choose the hyperplanes based on these support vectors!

Maximum Margin Classifier



Maximum Margin Classifier



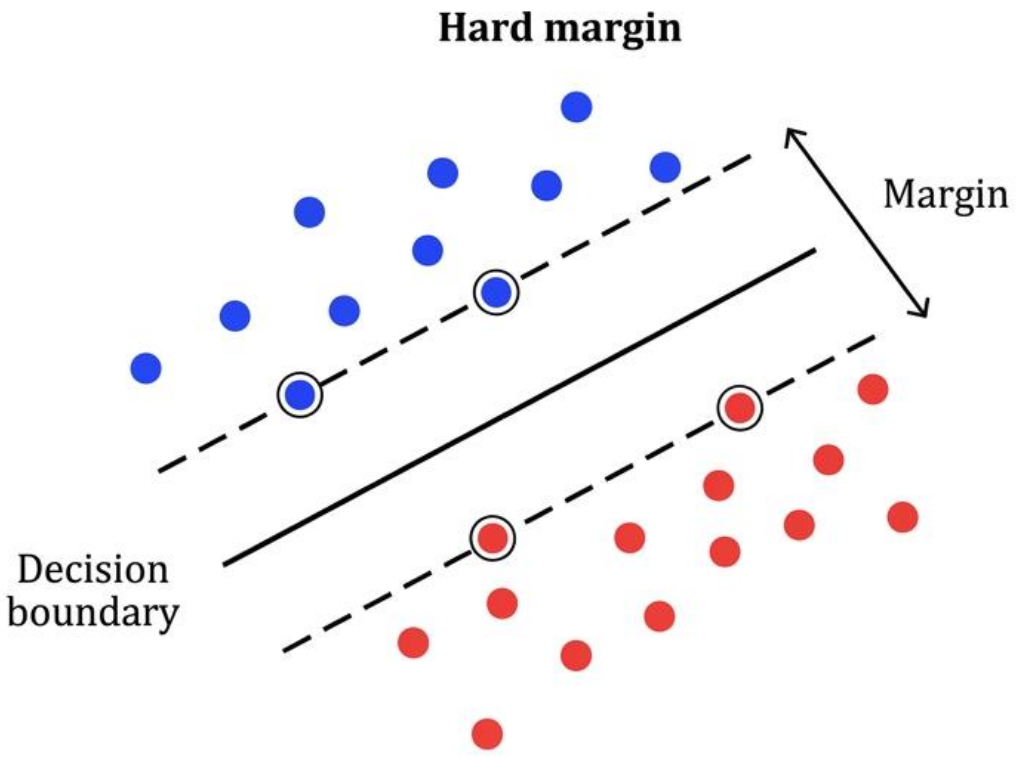
Support Vectors: These are data points in a dataset that are closest to the hyperplane and play a crucial role in determining the optimal hyperplane in SVM.

Hyperplane: In SVM, a hyperplane is a decision boundry that seperates the data points into different classes. The goal of SVM is to find the optimal hyperplane that maximizes the margin between the two classes.

Margin: The margin in SVM refers to the distance between the hyperplane and the closest data points from each class. The goal of SVM is th maximize this margin to increase the classifier's ability to generalize to new data.

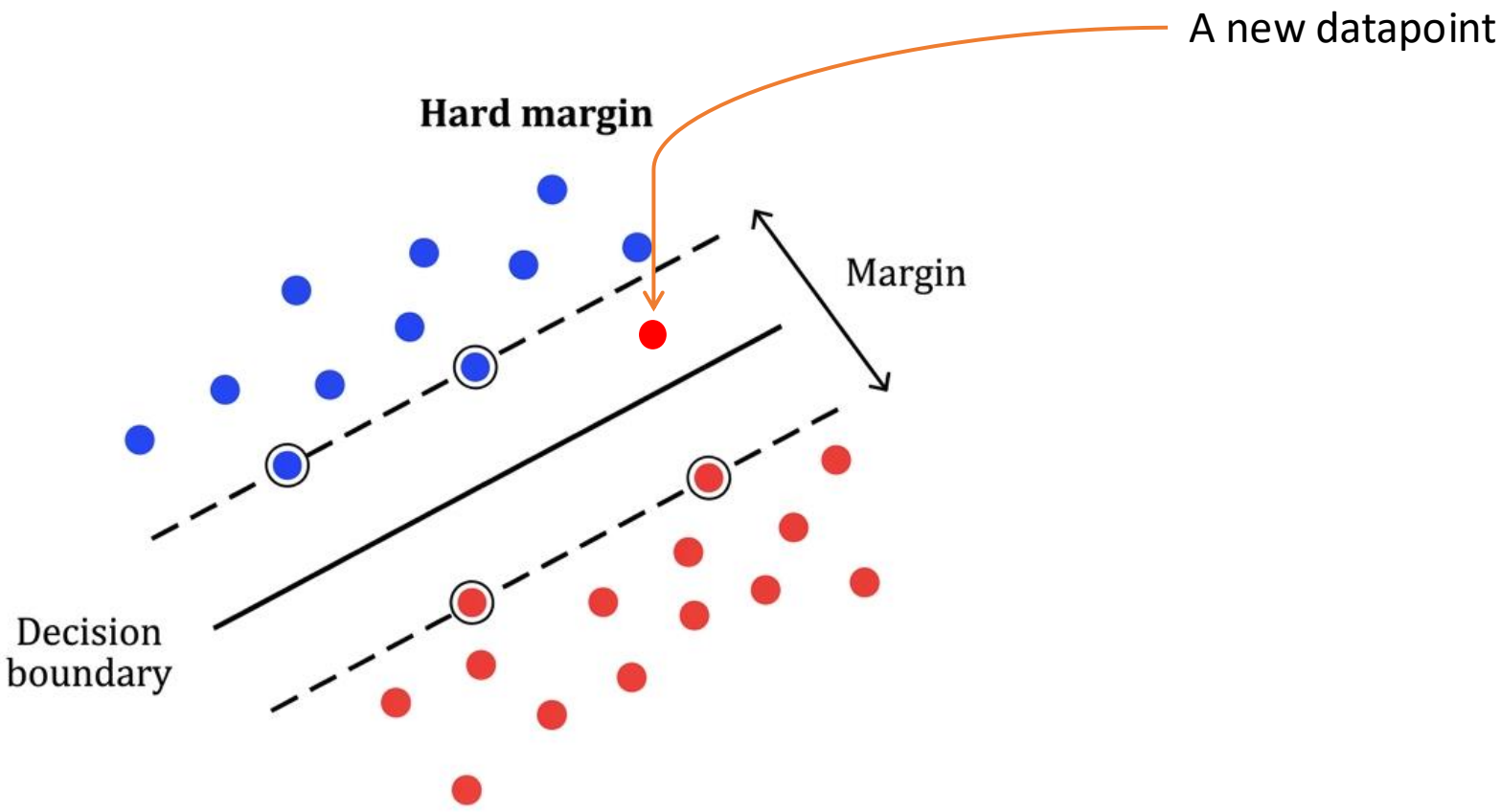
Issues with Maximum Margin Classifier

The Maximal Margin Classifier (MMC) utilizes a **hard margin**, requiring the data to be perfectly and linearly separable without exceptions.



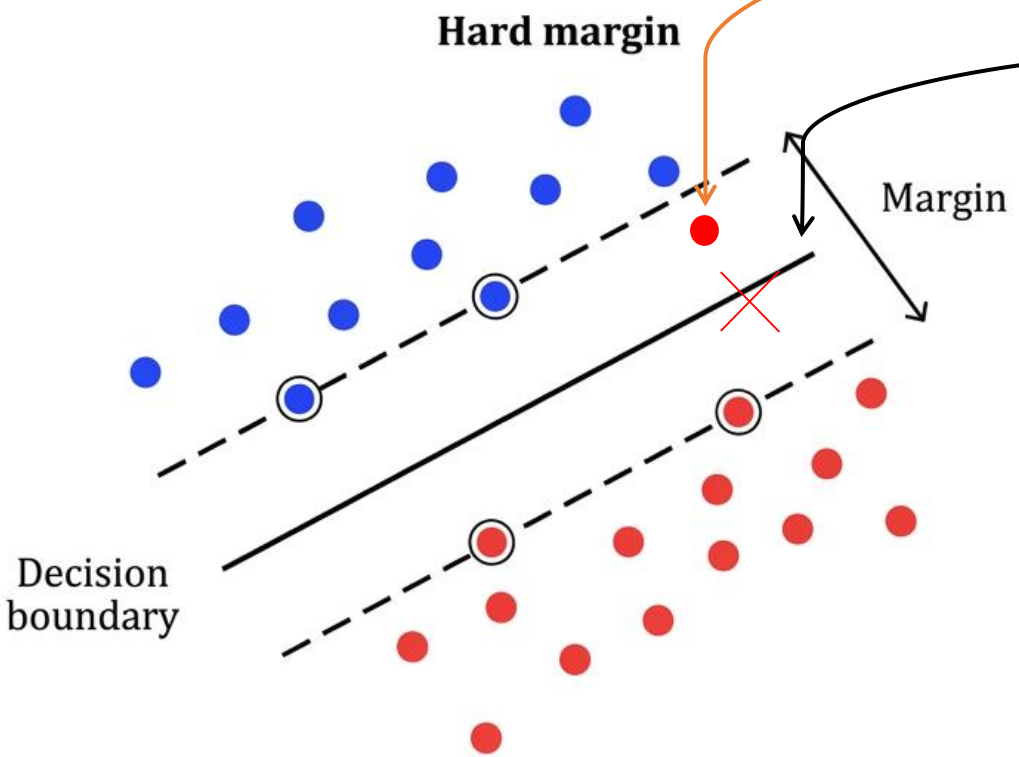
Issues with Maximum Margin Classifier

The Maximal Margin Classifier (MMC) utilizes a **hard margin**, requiring the data to be perfectly and linearly separable without exceptions.



Issues with Maximum Margin Classifier

The Maximal Margin Classifier (MMC) utilizes a **hard margin**, requiring the data to be perfectly and linearly separable without exceptions.

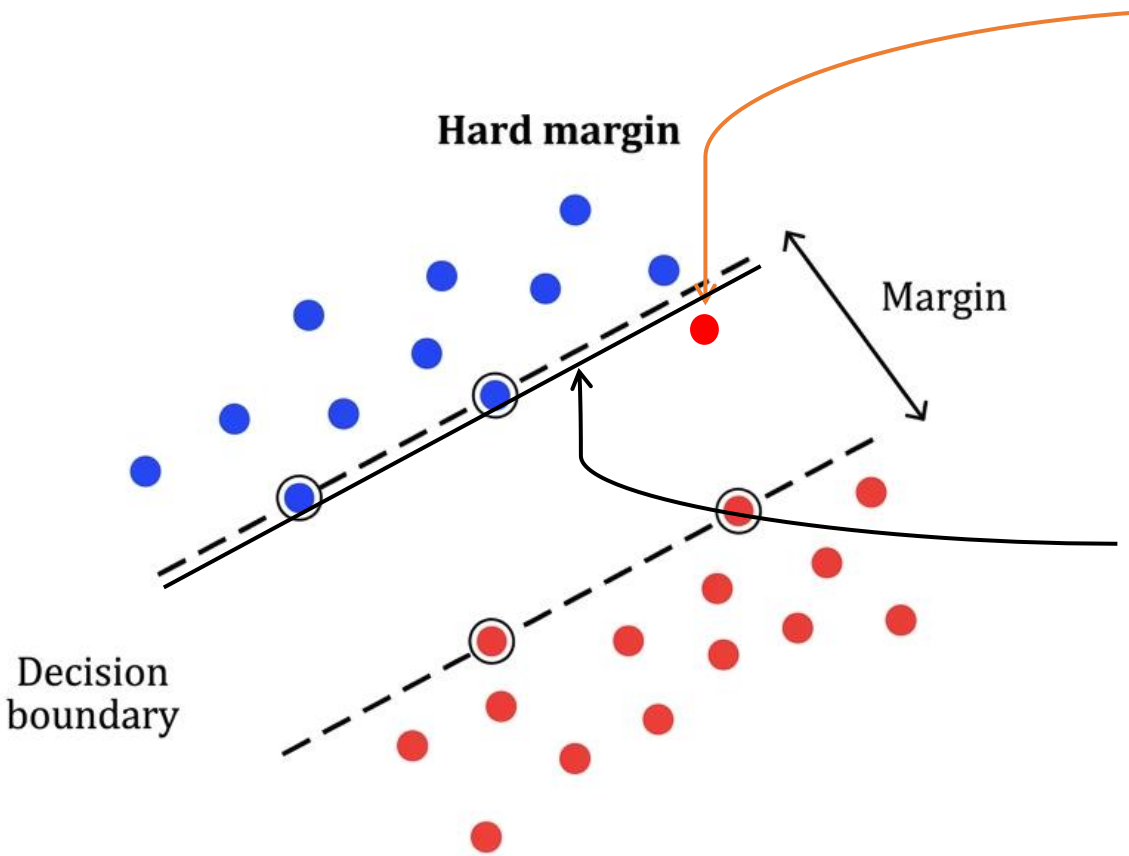


A new datapoint

The existing hyperplane is now invalid, as the dataset is no longer perfectly linearly separable.

Issues with Maximum Margin Classifier

The Maximal Margin Classifier (MMC) utilizes a **hard margin**, requiring the data to be perfectly and linearly separable without exceptions.



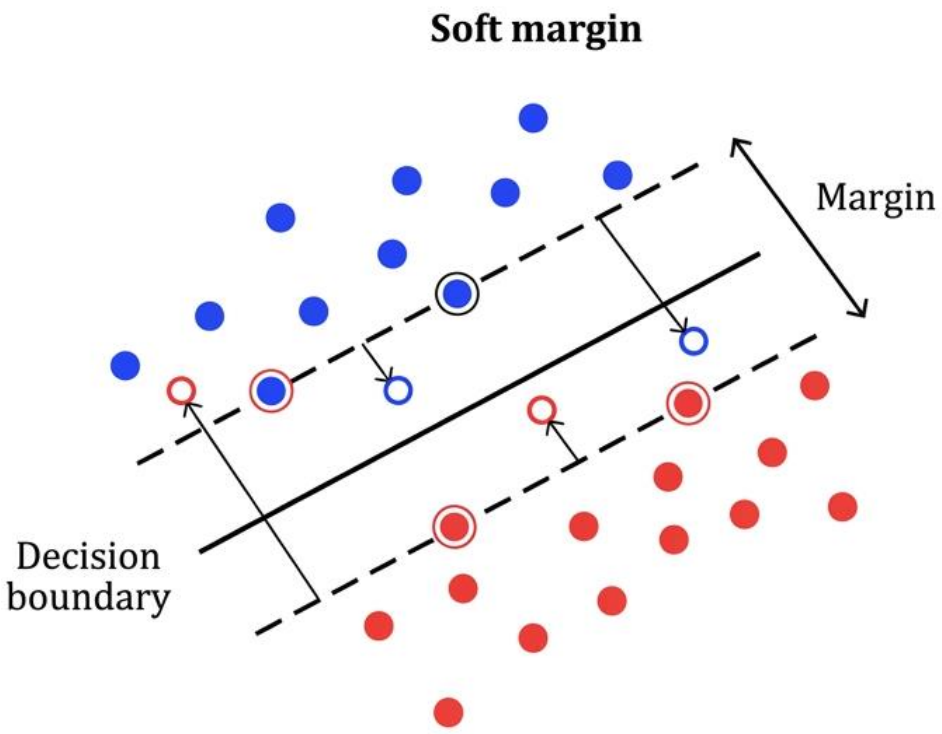
A new datapoint

The existing hyperplane is now invalid, as the dataset is no longer perfectly linearly separable.

This is the new hyperplane (to avoid misclassification)

SVM uses Soft Margin

In **Support Vector Machines (SVM)**, a **Soft Margin** is a modified version of the decision boundary that allows some data points to be misclassified or fall inside the margin.



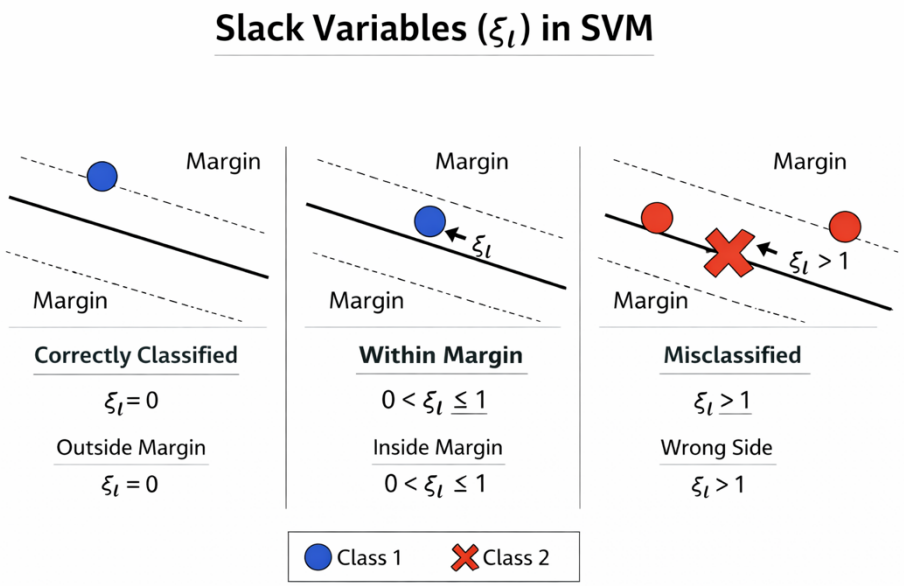
SVM uses Soft Margin

Slack Variables (ξ): To achieve a soft margin, we introduce "slack variables" for each data point. It measures **how much the margin constraint is broken**, not physical distance.

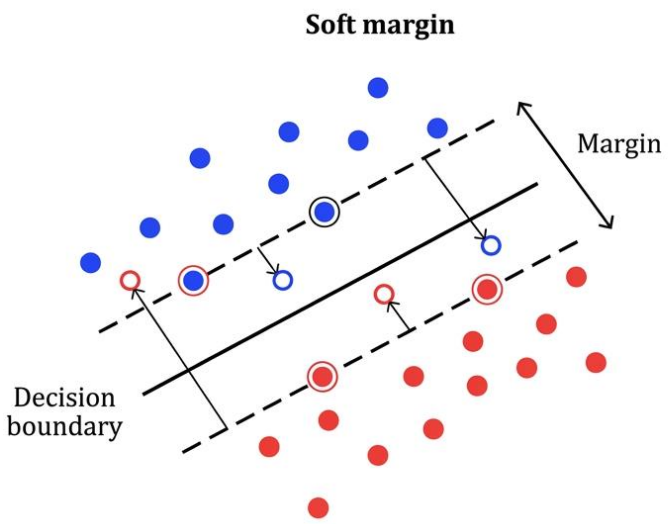
To allow margin violations, **slack variables** $\xi_i \geq 0$are introduced for each data point.

$\xi_i = 0$:Point is correctly classified and outside the margin
$0 < \xi_i \leq 1$:Point is inside the margin but correctly classified
$\xi_i > 1$:Point is misclassified

Slack variables quantify **how much each point violates the margin or decision boundary**.



SVM uses Soft Margin



1. **Margin term:** $\frac{1}{2}\|w\|^2$
   - Minimizing this **maximizes the margin**
   - Encourages a **simpler, smoother decision boundary**

2. **Error penalty term:** $C\sum \xi_i$
   - Penalizes margin violations and misclassifications
   - C controls **how costly errors are**

The Regularization Parameter (C): This is a user-defined hyperparameter that controls the trade-off between two goals:

1. **Maximizing the margin width** (making it as large as possible).
2. **Minimizing misclassifications** (keeping the slack variables small).

Mathematically, the optimization problem becomes:

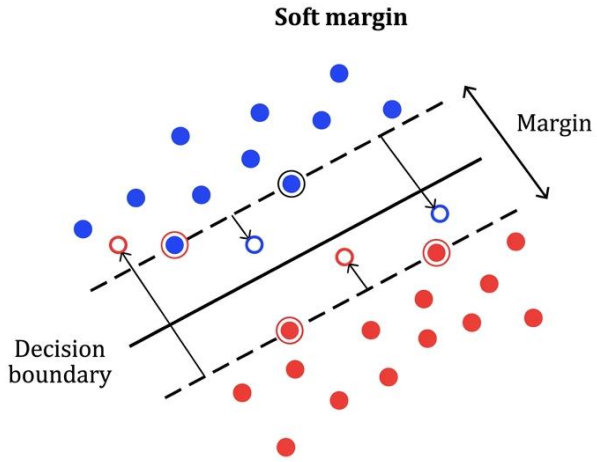
$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- w – weight vector (controls orientation of the hyperplane)
- b – bias (shifts the hyperplane)
- x_i – input data point
- $y_i \in \{-1, +1\}$ – class label
- ξ_i – *slack variable* (measures margin violation)
- C – regularization parameter (trade-off controller)
- n – number of training samples

SVM uses Soft Margin

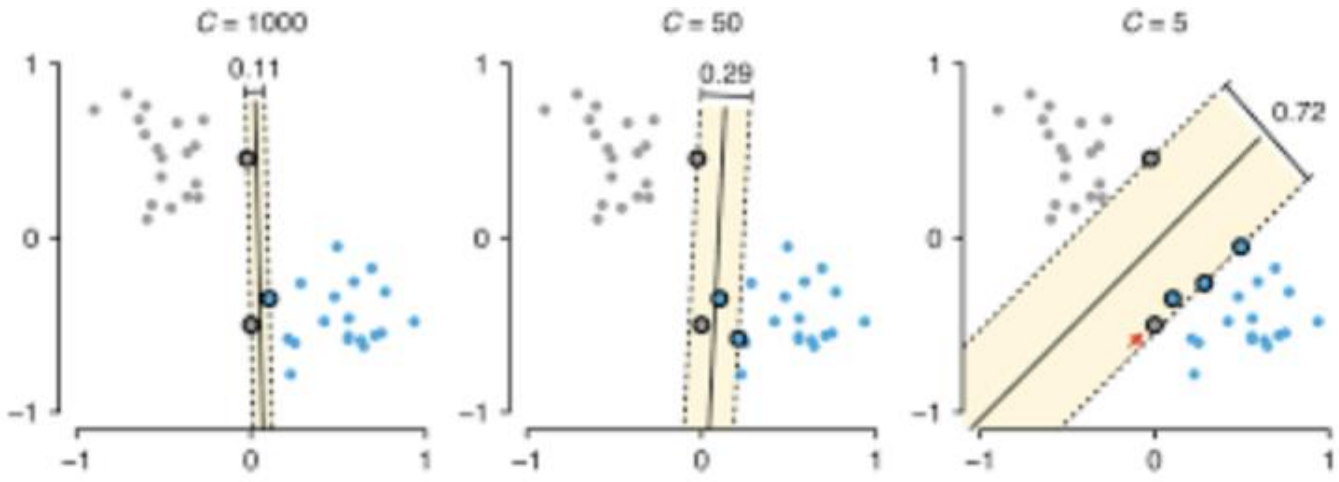


How C Affects the Margin

The value of C determines how "soft" the margin actually is:

Value of C	Meaning	Result
Small C	High tolerance for errors.	**Larger Margin**, but permits more misclassifications (High Bias, Low Variance).
Large C	Low tolerance for errors.	**Narrower Margin**, behaves more like a Hard Margin (Low Bias, High Variance).

Relation between Regularization Parameter C and SVM

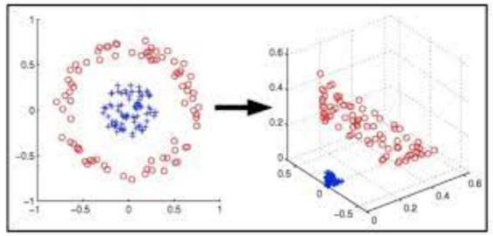


- As the value of C increases the margin decreases thus Hard SVM.
- If the values of C are very small the margin increases thus Soft margin VM.

Kernel trick in SVM

Support Vector Machine deals with nonlinear data by transforming it into a higher dimension where it is linearly separable.

Support Vector Machine does so by using different values of Kernel.



Linear Kernel

Suitable for linearly separable data, providing a straight decision boundary.

RBF Kernel

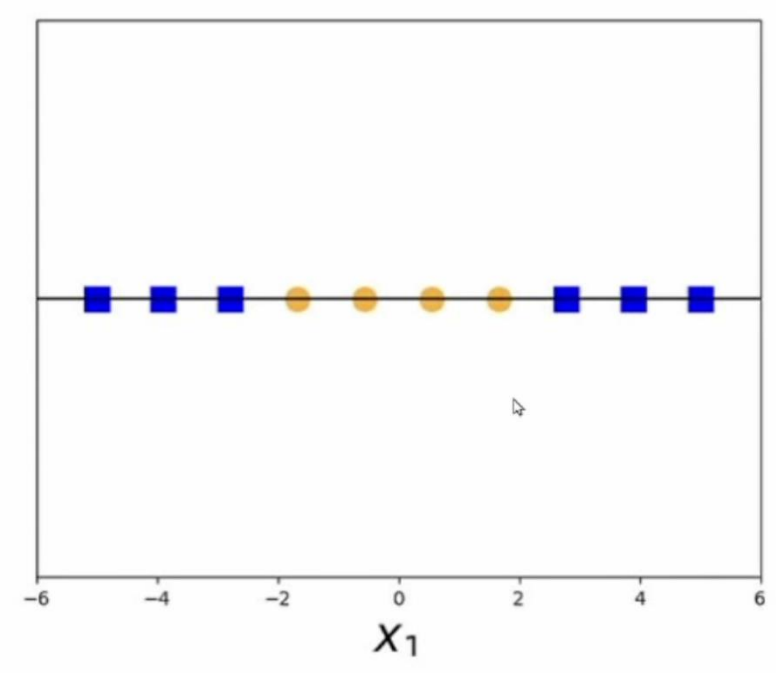
Effective for non-linear data, creating a curved decision boundary.

Poly Kernel

Also useful for non-linear data, generating a polynomial decision boundary.

Kernel trick in SVM

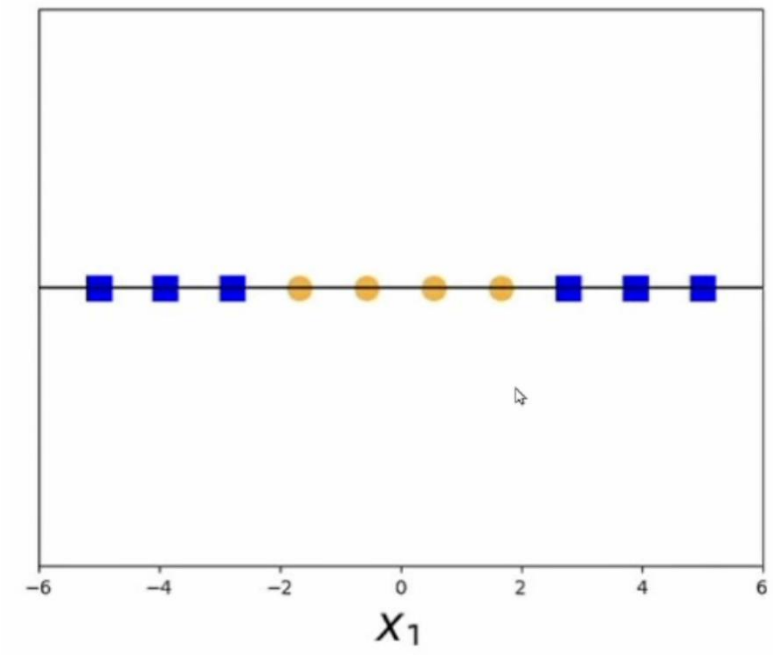
Example:2

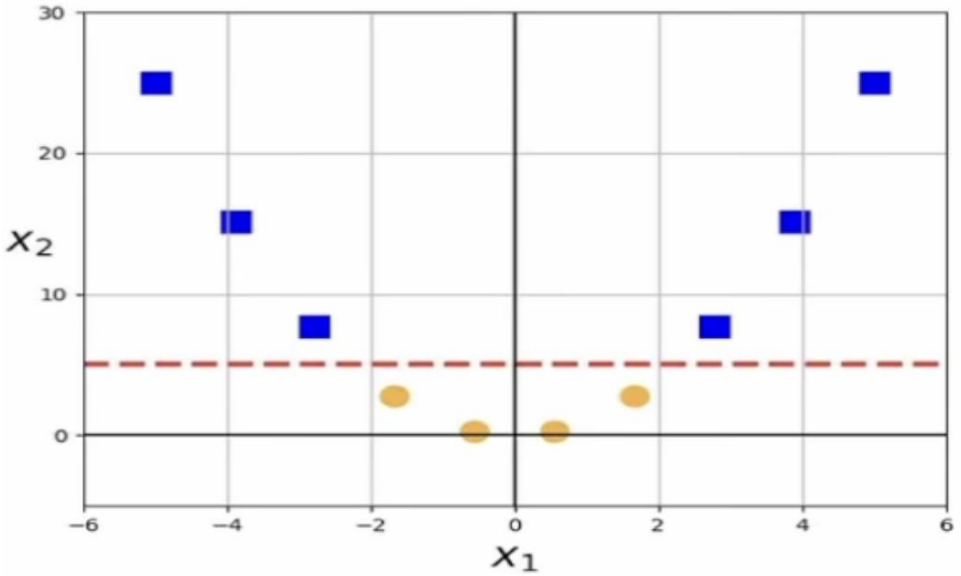


How do you draw the hyperplane?

Kernel trick in SVM

Example:2

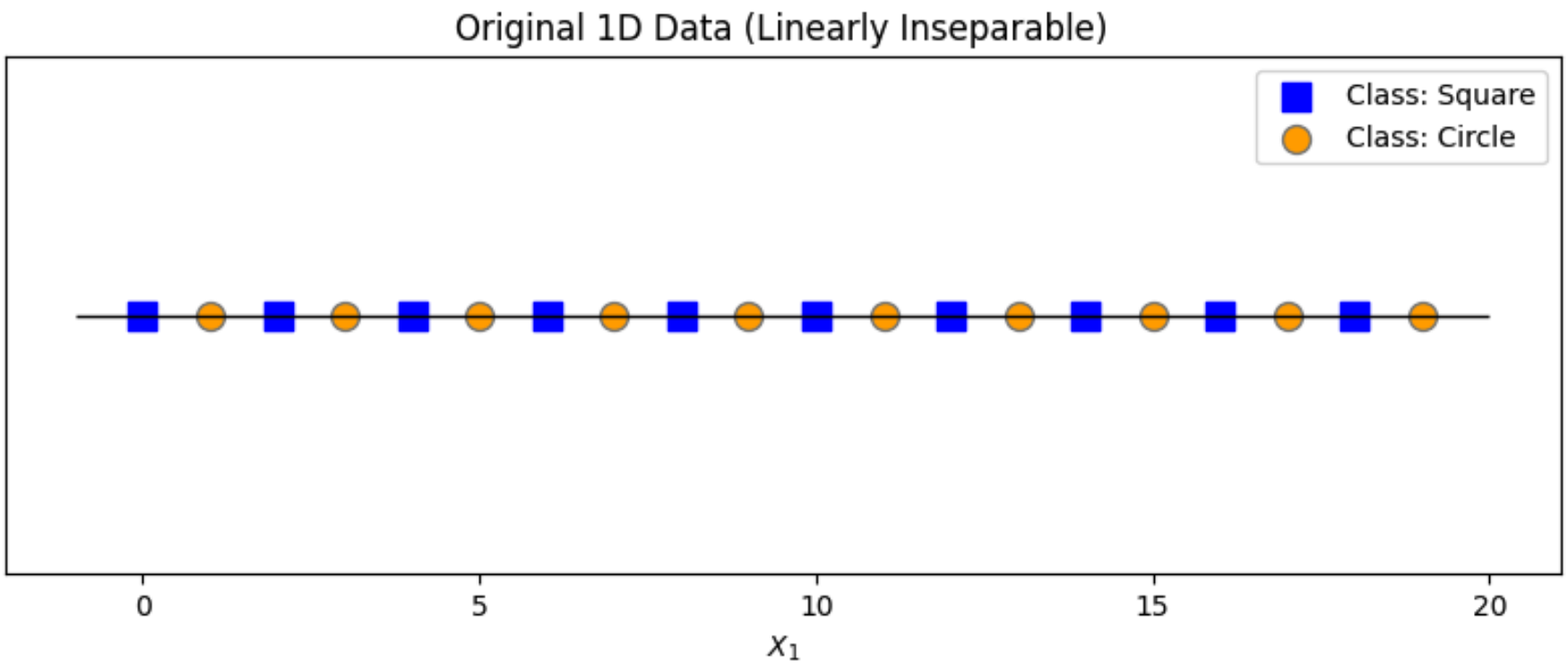




Kernal Trick : $x = x^2$

Kernel trick in SVM

Example:2



Kernal Trick : x = x mod 2

Kernel trick in SVM

Example:2

```
import numpy as np
import matplotlib.pyplot as plt


# 1. Generate the 1D data from the image (alternating classes)
x = np.arange(0, 20, 1)
# Class 0: Blue Squares (Even indices), Class 1: Orange Circles (Odd indices)
y_class = x % 2


# 2. Apply the transformation (Mapping to 2D)
# We use x for the horizontal axis and (x % 2) for the vertical axis
x_transformed = x % 2


# Create the plots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
plt.subplots_adjust(hspace=0.4)
```

Kernel trick in SVM

Example:2

```
# --- Plot 1: Before Kernel Trick (1D) ---
ax1.hlines(0, -1, 20, colors='black', linewidth=1)
ax1.scatter(x[y_class == 0], np.zeros_like(x[y_class == 0]),
c='blue', marker='s', s=100, label='Class: Square')
ax1.scatter(x[y_class == 1], np.zeros_like(x[y_class == 1]),
c='orange', marker='o', s=100, label='Class: Circle', edgecolors='gray')
ax1.set_title("Original 1D Data (Linearly Inseparable)")
ax1.set_xlabel("X_1")
ax1.set_yticks([])
ax1.legend()


# --- Plot 2: After Transformation (2D) ---
# We map x -> (x, x mod 2)
ax2.scatter(x[y_class == 0], x_transformed[y_class == 0],
c='blue', marker='s', s=100, label='Class: Square')
ax2.scatter(x[y_class == 1], x_transformed[y_class == 1],
c='orange', marker='o', s=100, label='Class: Circle', edgecolors='gray')
```

Kernel trick in SVM

Example:2

```
#Draw the decision boundary (Hyperplane in 2D)

ax2.axhline(0.5, color='red', linestyle='--', label='Decision Boundary')


ax2.set_title("Transformed 2D Data (Linearly Separable)")
ax2.set_xlabel("X_1")

ax2.set_ylabel(r"X_1 \mathrm{mod} 2")
ax2.legend()


plt.show()
```

Testing SVM with IRIS Dataset

Home Work